

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине Построение и анализ алгоритмов

Студент гр. 4343

Роменский М.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание

Требуется решить задачу коммивояжёра двумя способами:

- Точный метод — метод ветвей и границ, вариант 2: алгоритм Литтла с модификацией. После приведения матрицы к нижней оценке добавляется оценка остатка маршрута на основе минимального остовного дерева.
- Приближённый метод — алгоритм ближайшего соседа, то есть АБС. Обход необходимо начинать со стартовой вершины. В используемом формате входа стартовой вершиной является город 0.
- Требование: прохождение задания 3.3 на Stepik.

Описание алгоритма

Точный метод: алгоритм Литтла с модификацией (МОД-оценка)

Основная идея:

Задача коммивояжёра сводится к нахождению гамильтонова цикла минимального веса в заданном графе. Поскольку количество возможных маршрутов растёт как $(n-1)!$, прямой перебор становится неприменимым уже при $n > 10-12$. В связи с этим используется метод ветвей и границ (МВиГ), который позволяет существенно сократить пространство поиска за счёт отбрасывания заведомо неперспективных вариантов.

Алгоритм Литтла является одной из классических реализаций МВиГ применительно к задаче коммивояжёра. Его ключевая идея заключается в следующем: всё множество возможных решений делится на подмножества (ветви), для каждого из которых вычисляется нижняя граница стоимости. Если нижняя граница ветви оказывается не меньше стоимости уже найденного решения, эта ветвь отбрасывается без дальнейшего исследования.

В варианте 2 применяется модификация, при которой нижняя оценка формируется как максимум из двух независимых оценок:

1. Полусумма двух легчайших рёбер по кускам — отражает тот факт, что в итоговом цикле каждая вершина должна иметь ровно одно входящее и одно исходящее ребро.
2. Вес минимального остовного дерева (МОД) на кусках — отражает необходимость соединения всех кусков в единую замкнутую структуру.

Выбор максимального значения из двух оценок обеспечивает более сильную (более высокую) нижнюю границу, что повышает эффективность отсечения бесперспективных ветвей.

Понятие «куска»:

В рамках варианта 2 под куском понимается построенная на текущий момент цепочка вершин, которая войдёт в итоговое решение. На начальном этапе каждая вершина образует отдельный кусок из одного элемента. По мере того как алгоритм включает в решение дугу $u \rightarrow v$, кусок, оканчивающийся вершиной u , объединяется с куском, начинающимся вершиной v . В результате образуется более длинная цепочка. Количество кусков постепенно сокращается, пока все вершины не окажутся в единственной цепочке, представляющей собой полный гамильтонов цикл.

Допустимые дуги:

При расчёте нижних оценок рассматриваются исключительно допустимые дуги. Дуга $u \rightarrow v$ признаётся допустимой при одновременном выполнении двух условий:

- вершина u является концом некоторого куска;
- вершина v является началом другого куска.

Дуги, соединяющие вершины внутри одного куска, а также дуги, ведущие в уже пройденные вершины, в расчётах не участвуют.

Оценка 1: полусумма двух легчайших рёбер по кускам.

Для каждого куска рассматриваются:

- вес наименее затратной допустимой дуги, входящей в начало данного куска.
- вес наименее затратной допустимой дуги, исходящей из конца данного куска.

При наличии m кусков формируется набор из $2m$ чисел. Их сумма делится пополам, что и даёт искомую нижнюю оценку стоимости оставшихся рёбер, необходимых для замыкания всех кусков в единый цикл.

Почему делим на 2: каждая дуга в итоговом цикле одновременно является исходящей для одного куска и входящей для

другого. При суммировании по всем кускам каждая дуга учитывается дважды (один раз как исходящая, один раз как входящая), поэтому сумма делится на 2.

Оценка 2: вес МОД на кусках.

Строится минимальное остовное дерево, в котором:

- роль вершин играют куски;
- роль рёбер играют допустимые дуги, соединяющие конец одного куска с началом другого;
- вес каждого ребра равен стоимости соответствующей дуги.

Для построения МОД применяется алгоритм Краскала. Полученный вес дерева представляет собой минимально возможную стоимость соединения всех кусков в связную структуру. Поскольку любой гамильтонов цикл также соединяет все куски, его стоимость заведомо не меньше веса МОД, что делает эту величину корректной нижней границей.

Итоговая нижняя оценка:

Финальная нижняя граница вычисляется по формуле.

Приведение матрицы — стандартная процедура алгоритма Литтла: из каждой строки вычитается её минимальный элемент, затем аналогичная операция выполняется для столбцов. Совокупность вычтенных величин формирует самостоятельную нижнюю оценку стоимости продолжения маршрута.

Алгоритм работы:

1. Приведение матрицы. Из каждой строки вычитается минимальное значение, после чего аналогичная процедура выполняется для каждого столбца. Накопленная сумма вычетов добавляется к текущей нижней оценке. В результате в каждой строке и каждом столбце гарантированно присутствует хотя бы один нулевой элемент.

2. Расчёт нижних оценок. Для текущего набора кусков вычисляются:

- полусумма двух легчайших рёбер;
- вес МОД.

3. Из двух полученных значений выбирается наибольшее, которое прибавляется к оценке приведения.

4. Выбор дуги для ветвления. Среди нулевых элементов приведённой матрицы отыскивается дуга с максимальным штрафом. Штраф определяется как сумма минимальных элементов в соответствующих строке и столбце (без учёта самой выбранной клетки). Большой штраф означает, что отказ от включения данной дуги приведёт к существенному удорожанию решения, поэтому ветвление по ней обеспечивает наиболее эффективные отсечения.

5. Левая ветвь. Дуга $u \rightarrow v$ включается в решение. Строка u и столбец v исключаются из дальнейшего рассмотрения (из u выход уже сделан, в v вход уже выполнен). Дополнительно запрещается дуга, способная замкнуть преждевременный цикл: из конца цепочки, содержащей v , в начало цепочки, содержащей u . Куски, включающие вершины u и v , объединяются.

6. Правая ветвь. Дуга $u \rightarrow v$ исключается из рассмотрения (соответствующий элемент матрицы заменяется на бесконечность). Все прочие возможности сохраняются.

7. Отсечение. Ветвь отбрасывается, если её нижняя оценка не меньше стоимости наилучшего из уже найденных решений.

8. Завершение. Когда все вершины оказываются в единственном куске, цикл замыкается дугой, ведущей из конца цепочки в её начало. Если полученная стоимость оказывается ниже текущего рекорда, рекорд обновляется.

9. Восстановление маршрута. По зафиксированным в процессе работы дугам восстанавливается полная последовательность вершин, начиная со стартовой.

Правило запрета дуги (важное замечание варианта 2):

При включении дуги XU в левой ветви запрещению подлежит не обязательно обратная дуга YX , а та дуга, которая способна замкнуть преждевременный цикл. Конкретное правило зависит от структуры уже построенных цепочек:

- Если ранее была включена дуга AB и теперь включается дуга BC , то запретить следует дугу CA (она замкнула бы цикл $A \rightarrow B \rightarrow C \rightarrow A$).
- Если ранее были включены дуги AB и CD , а теперь включается дуга BC , то запретить следует дугу DA (она замкнула бы цикл $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$).

В реализации это достигается путём отслеживания начала и конца каждой цепочки: при включении дуги $u \rightarrow v$ запрещается дуга, идущая из конца цепочки, содержащей v , в начало цепочки, содержащей u .

Приближённый метод: АБС (алгоритм ближайшего соседа)

Основная идея:

Алгоритм ближайшего соседа представляет собой жадную стратегию, при которой на каждом шаге принимается локально наилучшее решение — выбирается ближайший непосещённый город. Такой подход обеспечивает высокую скорость работы, однако не гарантирует нахождения глобально оптимального маршрута.

Алгоритм работы:

1. Инициализация. Маршрут начинается со стартовой вершины. Текущей считается стартовая вершина. Все остальные вершины помечаются как непосещённые.

2. Выбор следующего города. Из множества непосещённых городов отбирается тот, переход в который из текущего города требует минимальных затрат.

3. Переход. Выбранный город помечается как посещённый и добавляется в маршрут. Текущей становится эта вершина. Суммарная стоимость маршрута увеличивается на вес пройденного ребра.

4. Повторение. Шаги 2–3 выполняются повторно до тех пор, пока не будут посещены все города (итого $n-1$ итераций).

5. Замыкание цикла. Из последней посещённой вершины выполняется переход обратно в стартовый город. Стоимость маршрута увеличивается на вес этого завершающего ребра.

6. Проверка допустимости. Если на каком-либо этапе отсутствует допустимый переход (все рёбра в непосещённые города имеют бесконечный вес) либо отсутствует обратное ребро в стартовую вершину, маршрут признаётся несуществующим.

Особенность варианта 2: АБС стартует строго из заданной стартовой вершины (в рамках данной работы — из города 0)..

Свойства АБС:

- Временная сложность: $O(n^2)$ — на каждом из n шагов выполняется просмотр всех n городов для выбора наилучшего кандидата.
- Потребление памяти: $O(n)$ — хранятся массив посещённых вершин и формируемый маршрут.
- Качество решения: в худшем случае результат может сколь угодно сильно отклоняться от оптимального, однако на практике алгоритм нередко даёт удовлетворительные приближения.
- Роль в точном методе: решение, полученное АБС, используется в качестве начальной верхней границы (initial upper

bound) для метода ветвей и границ, что позволяет сразу отсекаать значительную часть неперспективных ветвей.

Оценка сложности

Точный метод (Литтл + МОД-оценка)

Временная сложность:

В наихудшем сценарии алгоритм Литтла вынужден исследовать экспоненциальное число подмножеств дуг, что приводит к оценке $O(n^2 \cdot 2^n)$ операций. Данная оценка складывается из следующих факторов:

- 2^n — число возможных подмножеств вершин (представимых битовыми масками);
- n — количество вариантов последней вершины в подмножестве;
- n — количество вариантов следующей вершины для перехода.

На практике механизм отсечений по нижней оценке радикально сокращает число фактически исследуемых ветвей. Тем не менее, в вырожденных случаях (например, когда все веса рёбер одинаковы) отсечения теряют эффективность, и реальная сложность приближается к теоретическому пределу $O(n^2 \cdot 2^n)$.

Дополнительные вычислительные затраты на каждом шаге включают:

- приведение матрицы — $O(n^2)$;
- расчёт полусуммы — $O(n^2)$ (для каждого куска требуется просмотр всех остальных кусков);
- построение МОД — $O(n^2 \log n)$ (сортировка рёбер с последующим применением алгоритма Краскала).

Сложность по памяти:

В худшем случае $O(n^2 \cdot 2^n)$ — для хранения всех возможных состояний (матриц и конфигураций кусков) в приоритетной очереди. На практике благодаря механизму отсечений размер очереди остаётся существенно меньше теоретического максимума. Каждое хранимое состояние включает:

- приведённую матрицу размерности $n \times n$ — $O(n^2)$;
- описание набора кусков — $O(n)$;
- текущую стоимость и частичный маршрут — $O(n)$.

Приближённый метод (АБС)

Временная сложность:

$O(n^2)$ — на каждом из $n-1$ шагов просматриваются все n городов для поиска ближайшего непосещённого. Замыкание — $O(1)$.

Сложность по памяти:

$O(n)$ — для хранения массива посещённых вершин (visited), формируемого маршрута (path), указателя на текущую вершину и накопленной стоимости. Матрица весов занимает $O(n^2)$ памяти, однако она представляет собой входные данные и не относится к дополнительной памяти, потребляемой самим алгоритмом.

Тестирование:

Входные данные	Выходные данные	Комментарий
3; -1 2 5; 1 -1 3; 4 6 -1	Литтл: 0 1 2, стоимость 9.0; АБС: 0 1 2, стоимость 9.0	Минимальный нетривиальный случай, N=3, несимметричный полный граф
4; -1 2 9 10; 1 -1 6 4; 15 7 -1 8; 6 3 12 -1	Литтл: 0 2 3 1, стоимость 21.0; АБС: 0 1 3 2, стоимость 33.0	Несимметричная матрица, N=4, АБС попал в жадную ловушку
4; -1 3 1 7; 2 -1 4 5; 6 8 -1 2; 3 1 9 -1	Литтл: 0 2 3 1, стоимость 6.0; АБС: 0 2 3 1, стоимость 6.0	Несимметричная матрица, N=4, АБС совпал с оптимумом
4; -1 4 8 12; 4 -1 5 9; 8 5 -1 6; 12 9 6 -1	Литтл: 0 1 2 3, стоимость 27.0; АБС: 0 1 2 3, стоимость 27.0	Симметричная матрица, N=4, несколько оптимальных маршрутов
5; -1 10 15 20 25; 10 - 1 35 25 30; 15 35 -1 30 5; 20 25 30 -1 12; 25 30 5 12 -1	Литтл: 0 1 3 4 2, стоимость 67.0; АБС: 0 1 3 4 2, стоимость 67.0	Симметричная матрица, N=5, АБС совпал с оптимумом
4; -1 7 7 7; 7 -1 7 7; 7 7 -1 7; 7 7 7 -1	Литтл: 0 1 2 3, стоимость 28.0; АБС: 0 1 2 3, стоимость 28.0	Все рёбра равны 7, N=4, любой маршрут оптимален

6; -1 2 10 11 12 1; 2 -1 3 14 15 16; 10 3 -1 4 17 18; 11 14 4 -1 5 19; 12 15 17 5 -1 20; 1 16 18 19 20 -1	Литтл: 0 1 2 3 4 5, стоимость 35.0; АБС: 0 5 1 2 3 4, стоимость 41.0	Симметричная матрица, N=6, АБС хуже оптимума
4; -1 5 -1 -1; 5 -1 -1 -1; -1 -1 -1 3; -1 -1 3 -1	Литтл: no path; АБС: no path	Граф несвязный, две компоненты связности, нет гамильтонова цикла
1; -1	Литтл: 0, стоимость 0.0; АБС: 0, стоимость 0.0	Граничный случай: N=1, единственный город
2; -1 8; 8 -1	Литтл: 0 1, стоимость 16.0; АБС: 0 1, стоимость 16.0	Граничный случай: N=2
3; -1 -1 -1; -1 -1 -1; -1 -1 -1	Литтл: no path; АБС: no path	Все рёбра отсутствуют, маршрут невозможен

Вывод

В рамках данной лабораторной работы задача коммивояжёра была решена двумя способами: точным (алгоритм Литтла, дополненный оценкой на основе минимального остовного дерева) и приближённым (АБС — алгоритм ближайшего соседа).

Точный метод построен на схеме ветвей и границ. Для каждой ветви вычисляются две независимые нижние оценки: полусумма двух легчайших допустимых рёбер по кускам и вес минимального остовного дерева на кусках. В качестве итоговой оценки берётся наибольшее из двух значений, что обеспечивает более сильное отсечение неперспективных ветвей. Алгоритм корректно оперирует кусками (построенными цепочками), допустимыми дугами и правилом запрета дуги, замыкающей преждевременный цикл. Корректность реализации подтверждена сравнением с полным перебором на нескольких сотнях случайных тестов при N от 3 до 8 — во всех случаях результат точного метода совпал с оптимальным.

Приближённый метод (АБС) стартует из заданной вершины и на каждой итерации переходит в ближайший ещё не посещённый город. Алгоритм имеет квадратичную сложность и не гарантирует нахождение глобального минимума. На проведённых тестах АБС в ряде случаев давал оптимальное решение, в других — уступал точному методу.

Граничные ситуации (один или два города, несвязный граф, отсутствие гамильтонова цикла, несимметричная матрица) обрабатываются корректно и не приводят к ошибкам.